

อินเทอร์พรีเตอร์ไพธอนขนาดเล็กสำหรับเบราว์เซอร์ที่โปรแกรมได้บนโทรศัพท์มือถือ

A Small Python Interpreter for The Programmable Browser on Mobile Phones

นายกฤษณ์ชัย วันชัยนาวัน

นายณัฐวุฒิ นาไวย์

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต

ภาควิชาวิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ปีการศึกษา 2545

อินเทอร์พรีเตอร์ไพธอนขนาดเล็กสำหรับเบราว์เซอร์ที่โปรแกรมได้บนโทรศัพท์มือถือ

A Small Python Interpreter for The Programmable Browser on Mobile Phones

โดย

นายกฤษณ์ชัย วันชัยนาวัน

นายณัฐวุฒิ นาไวย์

อาจารย์ที่ปรึกษา

ดร. วิศิษฐ์ หิรัญกิตติ

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต

ภาควิชาวิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ปีการศึกษา 2545

ปริญญานิพนธ์ปีการศึกษา 2545

ภาควิชา วิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

เรื่อง อินเทอร์เน็ตไพธอนขนาดเล็กสำหรับเบราว์เซอร์ที่โปรแกรมได้บนโทรศัพท์มือถือ

A Small Python Interpreter for The Programmable Browser on Mobile Phones

ผู้จัดทำ

- | | |
|----------------------------|-----------------------|
| 1. นายกฤษณ์ชัย วันชัยนาวิน | รหัสประจำตัว 43015350 |
| 2. นายณัฐวุฒิ นาไวย | รหัสประจำตัว 43015357 |

อาจารย์ที่ปรึกษา

(ดร. วิศิษฐ์ หิรัญกิตติ)

อินเทอร์เน็ตไร้สายขนาดเล็กสำหรับบราวเซอร์ที่โปรแกรมได้บนโทรศัพท์มือถือ

นายกฤษณ์ชัย วันชัยนาวิน รหัส 43015350

นายณัฐวิภา นาไวย์ รหัส 43015357

ดร. วิศิษฐ์ หิรัญกิตติ อาจารย์ที่ปรึกษา

ปีการศึกษา 2545

บทคัดย่อ

การใช้งานเว็บบราวเซอร์ในปัจจุบันยังเป็นลักษณะทำที่ละขั้นตอนควบคุมโดยผู้ใช้ คือบราวเซอร์จะต้องคอยรอรับ URL จากผู้ใช้เสมอ ก่อนที่เปลี่ยน URL นั้นเป็นคำร้องขอส่งไปยังเว็บเซิร์ฟเวอร์เพื่อเปิดหน้าเว็บ ในบางครั้งผู้ใช้อ่าน URL ให้โดยตรง แต่บางครั้งเป็นทางอ้อมโดยการติดตามลิงค์ จึงได้มีแนวคิดในการสร้างบราวเซอร์ที่สามารถทำงานได้เองแบบกึ่งอัตโนมัติ โดยบราวเซอร์นี้จะทำงานตามชุดคำสั่งที่โปรแกรมให้โดยผู้ใช้ในรูปของสคริปต์หรือชุดคำสั่ง โดยจะเรียกบราวเซอร์ชนิดนี้ว่า “บราวเซอร์ที่สามารถโปรแกรมได้”

เพื่อให้บราวเซอร์สามารถกระทำคำสั่งในสคริปต์ได้เราจำเป็นต้องมีการสร้างอินเทอร์เน็ตพร็อกซีสำหรับประมวลผลคำสั่งนั้นๆ สำหรับปัญญาประดิษฐ์นี้เป็นการพัฒนาอินเทอร์เน็ตพร็อกซีให้กับบราวเซอร์ที่สามารถโปรแกรมได้สำหรับใช้บนเครื่องโทรศัพท์มือถือ โดยภาษาสคริปต์ที่เลือกใช้คือภาษาไพธอน เราจึงได้พัฒนาอินเทอร์เน็ตพร็อกซีขนาดเล็กขึ้นเพื่อการนี้

อินเทอร์เน็ตพร็อกซีที่พัฒนาในปัญญาประดิษฐ์นี้มีความสามารถในการแปล และกระทำชุดคำสั่งประโยคพื้นฐานภาษาไพธอน โดยมีองค์ประกอบของ ข้อมูลชนิดพื้นฐาน ตัวโอเปอเรเตอร์ ฟังก์ชันพื้นฐานในภาษา และประโยคเงื่อนไข ประโยคการทำซ้ำ แต่ไม่รวม การนิยามคลาสและฟังก์ชัน ทั้งนี้ได้มีการเพิ่มข้อมูลเกี่ยวกับเวลาเข้าไปในภาษาไพธอน สำหรับฟังก์ชันพื้นฐานในภาษาได้มีการเพิ่มฟังก์ชันการติดต่อสื่อสาร เช่น ฟังก์ชันการขอรับเอกสารผ่านทางเครือข่ายคอมพิวเตอร์โดยอาศัย URL ฟังก์ชันการส่งอีเมล เป็นต้น

A Small Python Interpreter for The Programmable Browser on Mobile Phones

Kritchai Vanchainavin

Nuttawuth Nawai

Dr. Visit Hirankitti Advisor

Abstract

So far the way we use a conventional web browser has been rather a manual one. The user must take control over the browser by sending his/her demanded URL to the browser as the browser's request to the web server repeatedly. Sometimes we do that directly by posing an explicit URL, but sometimes indirectly by following a link, which is in fact an implicit URL. To design a capable browser, we prefer it to work semi-automatically, rather than manually, by following a sequence of instructions programmed as a script by its user. The browser performs a task by executing the script given by the user. We call this kind of browser, a "programmable browser".

For such a browser to be able to execute instructions from the script, it needs to employ a script interpreter. In this project we have developed an interpreter for a programmable browser on a mobile phone. The language of the script is based on Python. We, therefore, have developed a small Python interpreter to use in a programmable browser.

The interpreter is able to interpret basic Python statements containing most basic data types, operators, built-in functions, and control structures, but not classes and function definitions. We also extend the Python language with a new data type, i.e. a time data type. The built-in functions of our language are mainly communicating functions, e.g. functions for getting a document from the Net referred by a URL, functions for sending emails, and so on.

กิตติกรรมประกาศ

ปริญญานิพนธ์ฉบับนี้คงไม่อาจเสร็จได้ด้วยดี หากไม่ได้รับความช่วยเหลือ และความร่วมมือจากหลายๆ ฝ่ายด้วยกัน บุคคลแรกที่ต้องกล่าวถึงเพราะเป็นส่วนสำคัญที่ทำให้วิทยานิพนธ์นี้เสร็จลงได้ก็คือ อาจารย์ วิชาญ หิรัญกิตติ อาจารย์ที่ปรึกษาปริญญานิพนธ์ ที่ให้ความเอาใจใส่ แนะนำ และช่วยเหลือทั้งทางด้านความรู้ และเครื่องมืออุปกรณ์ต่างๆ ที่จำเป็นต้องใช้ประกอบการทำปริญญานิพนธ์ฉบับนี้ ซึ่งต้องขอขอบพระคุณเป็นอย่างมาก

และต้องขอขอบพระคุณบุคคลสำคัญที่สุดที่ทำให้มีวันนี้ ซึ่งก็คือ บิดา มารดา อันเป็นที่เคารพรักยิ่ง ซึ่งได้เลี้ยงดูมาเป็นอย่างดี พร้อมทั้งให้โอกาสในการศึกษาอย่างเต็มที่ และยังให้กำลังใจ เอาใจใส่เสมอมา ในทุกๆ ด้านอันหาที่เปรียบมิได้ คณะผู้จัดทำขอระลึกในพระคุณอันสุดประมาณ และขอกราบขอบพระคุณมา ณ ที่นี้

กฤษณ์ชัย วันชัยนาวิน

ณัฐวุฒิ นาไวย์

สารบัญ

	หน้าที่
บทคัดย่อภาษาไทย	I
บทคัดย่อภาษาอังกฤษ	II
กิตติกรรมประกาศ	III
สารบัญ	IV
สารบัญตาราง	VII
สารบัญรูปภาพ	VIII
บทที่ 1 บทนำ	
1.1 ความสำคัญและที่มา	1
1.2 วัตถุประสงค์	2
1.3 ขอบเขตของงานวิจัย	2
1.4 วิธีการดำเนินงาน	2
บทที่ 2 ภาษาไพธอน (Python)	
2.1 คุณสมบัติของภาษาไพธอน	4
2.2 ชนิดของข้อมูล	6
2.2.1 ตัวเลข (Numeric)	7
2.2.2 จำนวนทางตรรกะ (Boolean)	7
2.2.3 สายอักขระ (String)	7
2.2.4 ลิสต์ (Lists)	8
2.2.5 ทUPLE (Tuples)	8
2.2.6 ดิกชันนารี (Dictionary)	8
2.2.7 ข้อมูลเปล่า (None)	9
2.3 ตัวกระทำ(Operators)	9
2.3.1 ตัวกระทำทางตรรกะ (Logical operators)	9
2.3.2 ตัวกระทำทางการเปรียบเทียบ (Comparison operators)	9
2.3.3 ตัวกระทำทางบิตไวส์ (Bitwise operators)	9
2.3.4 ตัวกระทำทางคณิตศาสตร์ (Arithmetic-Style operators)	10
2.3.5 ลำดับของตัวกระทำ (Precedence)	10
2.4 คำสงวน (Reserved words)	10
2.5 สโคปของโค้ด (Code block)	11
2.6 คอมเมนต์ (Comment)	11
2.7 การพิมพ์ statement เดี่ยวในหลายบรรทัด (Continuation)	11
2.8 การพิมพ์หลาย statement ในบรรทัดเดียว	11

2.9 การกำหนดค่าให้ตัวแปร (Variable assignment)	12
2.9.1 การกำหนดค่าหนึ่งค่าให้กับหลายตัวแปร (Multiple assignment)	12
2.9.2 การกำหนดค่าหลายค่าให้กับหลายตัวแปร	12
2.10 คำสั่งควบคุมการทำงาน (Control flow)	12
2.10.1 คำสั่ง IF	12
2.10.2 คำสั่ง WHILE	13
2.10.3 คำสั่ง FOR	13
2.11 การจัดการหน่วยความจำ (Memory management)	14
2.11.1 การประกาศตัวแปร (Variable Declarations)	14
2.11.2 การจองหน่วยความจำ (Memory Allocation)	14
2.11.3 การรวบรวมหน่วยความจำที่ไม่ได้ใช้ (Garbage Collection)	14
2.11.4 การนับการอ้างอิง (Reference counting)	15
2.11.5 คำสั่ง del	15
2.11.6 การลดค่า reference count	15
2.12 การใช้งานภาษาไพธอน	15
บทที่ 3 อินเทอร์เน็ต	
3.1 การFETCHคำสั่ง (Fetch)	17
3.2 การวิเคราะห์ข้อมูล (Analyze)	18
3.2.1 การตรวจจับคำ (Lexical Analysis)	18
3.3 การตรวจสอบประโยค (Syntax Analysis)	19
3.3.1 non terminal	20
3.3.2 terminal	20
3.4 พาร์ซทรี (Parse tree)	20
3.5 การกระทำตามคำสั่ง (Execution)	22
บทที่ 4 การออกแบบและการพัฒนาโปรแกรม	
4.1 การออกแบบและสร้างส่วนของการอ่านคำสั่ง	24
4.2 การออกแบบและสร้างส่วนของการตรวจจับคำและการตรวจสอบประโยค	24
4.2.1 การออกแบบ เรกกูลาร์เอ็กเพรสชัน	24
4.2.2 การออกแบบ บีเอ็นเอฟ	28
4.2.3 การออกแบบเรกกูลาร์เอ็กเพรสชันและบีเอ็นเอฟ สำหรับทำงานร่วมกับ บราวเซอร์ที่สามารถโปรแกรมได้	30
4.2.4 การออกแบบและสร้างพาร์ซทรี	33
4.3 การออกแบบและสร้างส่วนของการกระทำตามคำสั่ง	37
4.3.1 ตรวจสอบความผิดพลาดของชนิดตัวแปรที่นำมากระทำตามคำสั่ง	37

4.3.2 การกระทำตามคำสั่งที่รับเข้ามา	38
4.3.3 จัดการกับตัวแปรต่างๆ ที่ต้องนำไปกระทำหรือได้จากการกระทำ	38
บทที่ 5 ผลการทดสอบ	
5.1 การทดสอบความถูกต้องของกลุ่มคำที่ได้ทำการกำหนดโดยใช้เรกกูลาร์เอ็กเพรสชัน	39
5.2 การทดสอบความถูกต้องของประโยคที่ได้ทำการกำหนดโดยใช้บีเอ็นเอฟ	42
5.3 การทดสอบความถูกต้องของพารซ์ซรี	44
5.4 การทดสอบความถูกต้องของการกระทำตามคำสั่ง และนำไปใช้งานบนมือถือ	47
บทที่ 6 บทวิจารณ์และสรุป	
6.1 บทสรุป	51
6.2 ข้อดีของอินเตอร์พรีเตอร์ไพธอนขนาดเล็ก	51
6.3 ข้อเสียของอินเตอร์พรีเตอร์ไพธอนขนาดเล็ก	51
6.4 แนวทางการพัฒนา	52
ภาคผนวก ก. – การติดตั้งและใช้งานภาษาไพธอน	53
ภาคผนวก ข. – ไวยากรณ์ของภาษาไพธอน (Python Grammar)	55
บรรณานุกรม	61

สารบัญตาราง

	หน้าที่
ตารางที่ 2-1 ตารางเปรียบเทียบคุณสมบัติต่างๆ ของภาษาไพธอนกับภาษาอื่นๆ	6
ตารางที่ 3-1 ตารางตัวอย่างเรกกูลาร์เอ็กซ์เพรสชันของภาษาไพธอน	19
ตารางที่ 3-2 ตารางตัวอย่างบีเอ็นเอฟของภาษาไพธอน	20
ตารางที่ 4-1 ตารางเรกกูลาร์เอ็กซ์เพรสชันที่มีลักษณะคงที่ที่ได้ทำการออกแบบ	25
ตารางที่ 4-2 ตารางเรกกูลาร์เอ็กซ์เพรสชันที่สามารถเปลี่ยนแปลงได้ที่ได้ทำการออกแบบ	27
ตารางที่ 4-3 ตารางบีเอ็นเอฟบางส่วน of ภาษาไพธอนที่ได้ทำการออกแบบ	29
ตารางที่ 4-4 ตารางเรกกูลาร์เอ็กซ์เพรสชันสำหรับข้อมูลชนิดไทม์	30
ตารางที่ 4-5 ตารางการกระทำลักษณะต่างๆ สำหรับข้อมูลชนิดไทม์	31
ตารางที่ 4-6 ตารางบีเอ็นเอฟของคำสั่งการเปิดเว็บไซต์	32
ตารางที่ 4-7 ตารางบีเอ็นเอฟของคำสั่งการส่งเมลล์	32

สารบัญรูปภาพ

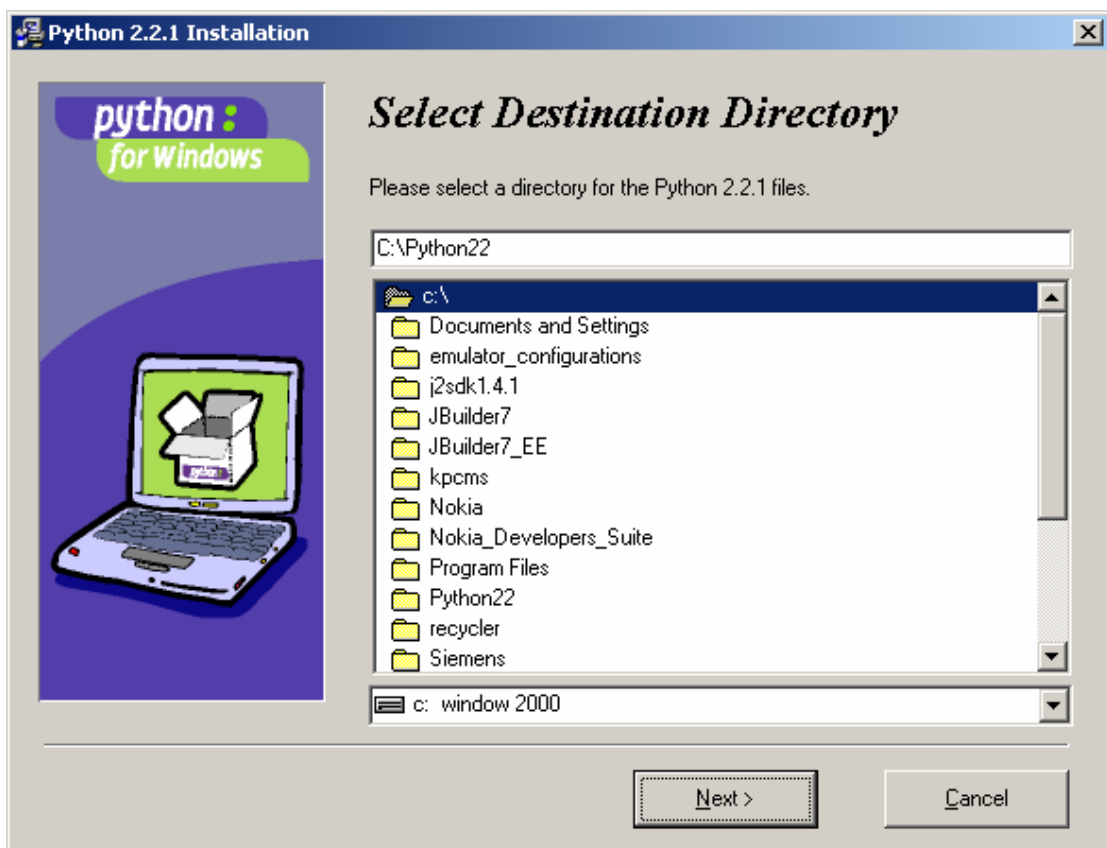
	หน้าที่
รูปที่ 2-1 แสดงโปรแกรมไพธอน IDLE	16
รูปที่ 2-2 แสดงโปรแกรมไพธอนคอนโซล	16
รูปที่ 3-1 แสดงส่วนประกอบหลักของอินเตอร์พรีตเตอร์	17
รูปที่ 3-2 แสดงพาร์ซทรีของคำสั่งทั่วไปของภาษาไพธอน	21
รูปที่ 3-3 แสดงพาร์ซทรีของคำสั่งที่มีนัยสำคัญของการกระทำไม่เท่ากัน	21
รูปที่ 4-1 แสดงองค์ประกอบของอินเตอร์พรีตเตอร์ไพธอนขนาดเล็ก	23
รูปที่ 4-2 แสดงพาร์ซทรีแบบสองกึ่ง	33
รูปที่ 4-3 แสดงพาร์ซทรีแบบสามกึ่ง	33
รูปที่ 4-4 แสดงตัวอย่างการสร้างทรีแบบสองกึ่งจากคำสั่งไพธอน $x + 5$	33
รูปที่ 4-5 แสดงตัวอย่างการสร้างทรีแบบสองกึ่งจากคำสั่งไพธอน $z = x * 5$	34
รูปที่ 4-6 แสดงตัวอย่างการสร้างทรีแบบสองกึ่งจากคำสั่งไพธอน <code>print x + 10</code>	34
รูปที่ 4-7 แสดงตัวอย่างการสร้างทรีแบบสองกึ่งจากคำสั่งการเปิดเว็บไซต์	35
รูปที่ 4-8 แสดงตัวอย่างการสร้างทรีแบบสามกึ่งจากคำสั่งไพธอน <code>if x < 10 : y = 10 else : y = 200</code>	35
รูปที่ 4-9 แสดงตัวอย่างการสร้างทรีแบบสามกึ่งจากคำสั่งไพธอนที่ซ้อนหลายชั้น	36
รูปที่ 5-1 แสดงตัวอย่างการทดสอบเรกกูลาร์เอ็กซ์เพรสชันของข้อมูลชนิด integer	39
รูปที่ 5-2 แสดงตัวอย่างการทดสอบเรกกูลาร์เอ็กซ์เพรสชันของข้อมูลชนิด string	40
รูปที่ 5-3 แสดงตัวอย่างการทดสอบเรกกูลาร์เอ็กซ์เพรสชันของค่าสว่นต่างๆ	40
รูปที่ 5-4 แสดงตัวอย่างการทดสอบเรกกูลาร์เอ็กซ์เพรสชันของตัวกระทำแบบต่างๆ	41
รูปที่ 5-5 แสดงตัวอย่างการทดสอบค่าที่ไม่ถูกต้องตามเรกกูลาร์เอ็กซ์เพรสชันใดเลย	41
รูปที่ 5-6 แสดงตัวอย่างการทดสอบประโยคที่เป็นคำสั่งแบบตามลำดับทั่วไป	42
รูปที่ 5-7 แสดงตัวอย่างการทดสอบประโยคที่เป็นคำสั่งแบบเงื่อนไขและแบบวนรอบ	43
รูปที่ 5-8 แสดงตัวอย่างการทดสอบประโยคที่ไม่ถูกต้องตามบีเอ็นเอฟ	43
รูปที่ 5-9 แสดงตัวอย่างการทดสอบพาร์ซทรีของประโยค $x + 5$	44
รูปที่ 5-10 แสดงตัวอย่างการทดสอบพาร์ซทรีของประโยค $x = y - z$	45
รูปที่ 5-11 แสดงตัวอย่างการทดสอบพาร์ซทรีของประโยค <code>if x == z : y = z1</code>	46
รูปที่ 5-12 แสดงตัวอย่างการป้อนสคริปต์การกำหนดค่าแล้วแสดงออกทางหน้าจอ	47
รูปที่ 5-13 แสดงผลลัพธ์จากการอินเตอร์พรีตสคริปต์การกำหนดค่าแล้วแสดงออกทางหน้าจอ	48
รูปที่ 5-14 แสดงตัวอย่างการป้อนและผลลัพธ์จากสคริปต์วนลูปและแสดงค่า	49
รูปที่ 5-15 แสดงตัวอย่างการป้อนสคริปต์ที่มีความผิดพลาด	50

ภาคผนวก ก.

การติดตั้งและใช้งานภาษาไพธอน

ขั้นตอนการติดตั้งโปรแกรม python 2.2.1

1. ดาวน์โหลดชุดติดตั้งโปรแกรมไพธอนได้จาก www.python.org โดยไม่ต้องเสียค่าใช้จ่ายใดๆ ซึ่งจะ
ได้ไฟล์ Python-2.2.1.exe ซึ่งเป็นไฟล์ที่ใช้ติดตั้งภาษาไพธอนเวอร์ชัน 2.2.1 บนระบบปฏิบัติการ
วินโดวส์
2. ทำการรันไฟล์ Python-2.2.1.exe เพื่อทำการติดตั้งโดยจะแสดงหน้าจอดังนี้



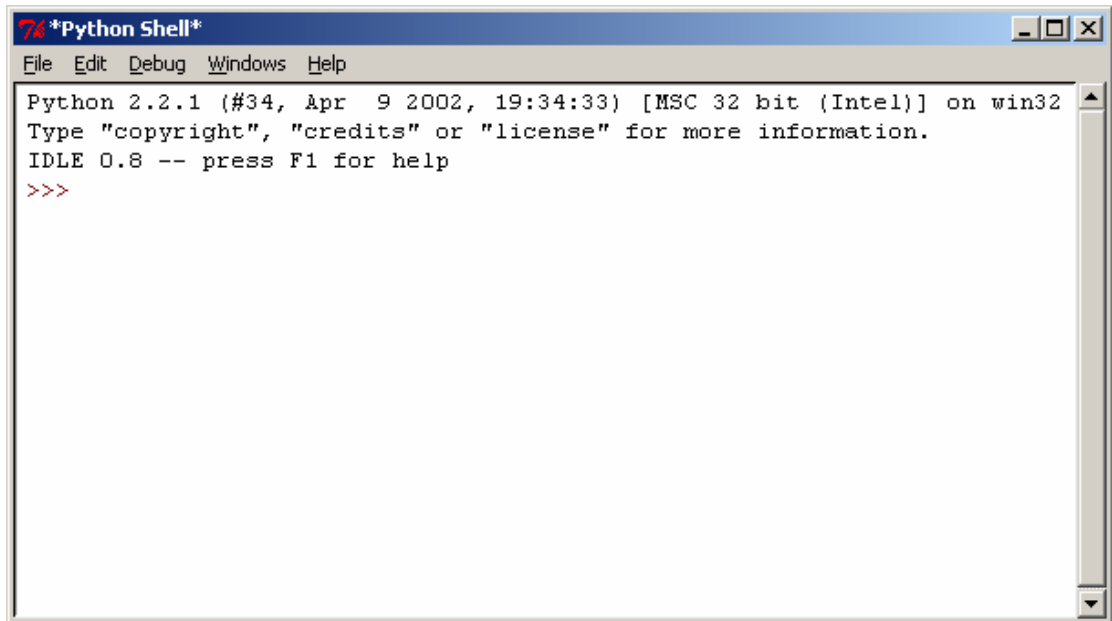
รูปที่ ก-1 แสดงหน้าต่างการติดตั้งโปรแกรมไพธอนเวอร์ชัน 2.2.1

3. ทำการเลือกไดเรกทอรีที่ต้องการจะติดตั้ง และปฏิบัติตามขั้นตอนเหมือนการลงโปรแกรมต่างๆ ไปก็
จะเป็นอันเสร็จการติดตั้งโปรแกรมไพธอนเวอร์ชัน 2.2.1

การใช้งานภาษาไพธอน

เนื่องจากโครงงานวิจัยนี้เป็นการพัฒนาอินเตอร์พรีเตอร์ที่อ้างอิงภาษาไพธอนเป็นส่วนใหญ่ ควรจะเข้าใจการใช้งานของภาษาไพธอนบ้างเล็กน้อย โดยจะใช้เพื่อทดลองคำสั่งต่างๆ ของภาษาไพธอน ซึ่งมีวิธีการใช้งานดังนี้

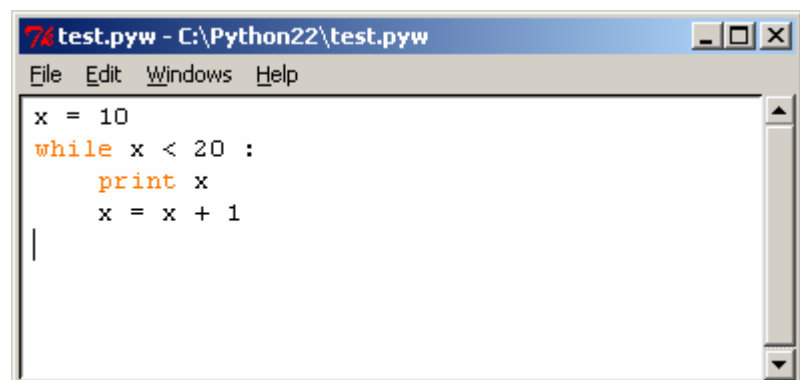
1. คลิกที่ Start → Programs → Python 2.1 → IDLE (Python GUI) ซึ่งจะปรากฏหน้าต่างดังรูป ก-2



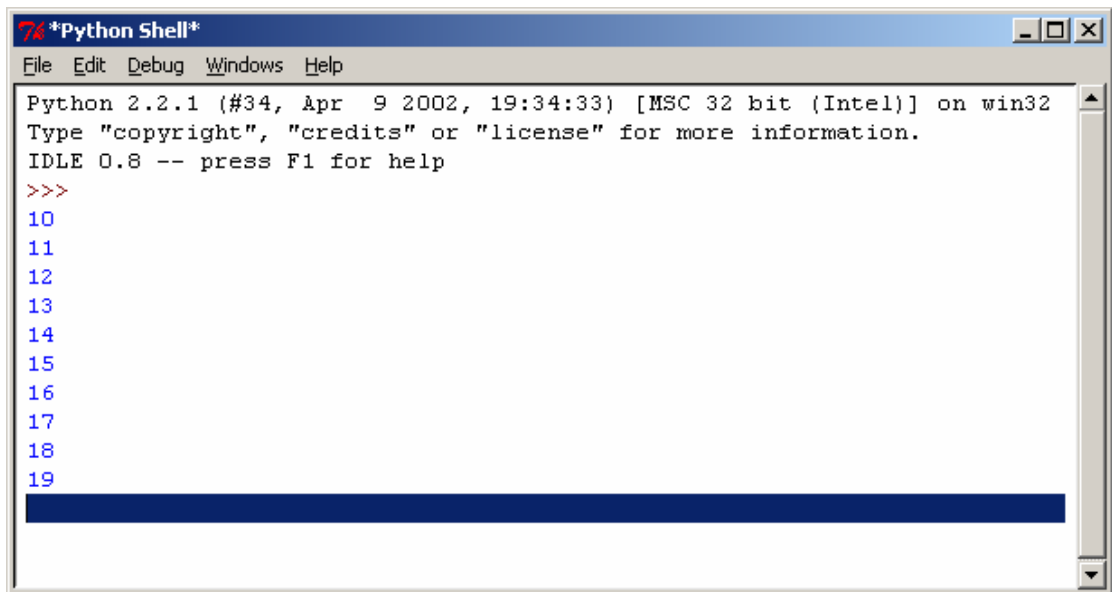
รูปที่ ก-2 แสดงหน้าต่างโปรแกรมไพธอน 2.2.1

2. สามารถที่จะทำการพิมพ์คำสั่งใดๆ ก็ได้ในภาษาไพธอน โดยโปรแกรมจะทำการอินเตอร์พรีตทันที
3. สามารถที่จะนำไฟล์สคริปต์ที่เขียนไว้แล้วมาทำการทดสอบก็ได้ โดยเลือกที่เมนู File → Open... แล้วทำการเลือกสคริปต์ที่ต้องการทำการรัน ซึ่งจะปรากฏหน้าจอ ดังรูปที่ ก-3

รูปที่ ก-3 แสดงหน้าต่าง
การเขียนสคริปต์เพื่อทำ
การรัน



4. หากต้องการทำการรันสคริปต์ที่เขียนไว้แล้วสามารถทำได้โดยการเลือกที่เมนู Edit → Run script ซึ่งหากทดลองรันสคริปต์จากรูป ก-3 โปรแกรมก็จะทำการอินเทอร์พรีตแล้วแสดงผลดังรูปที่ ก-4



```
*Python Shell*
File Edit Debug Windows Help
Python 2.2.1 (#34, Apr 9 2002, 19:34:33) [MSC 32 bit (Intel)] on win32
Type "copyright", "credits" or "license" for more information.
IDLE 0.8 -- press F1 for help
>>>
10
11
12
13
14
15
16
17
18
19
```

รูปที่ ก-4 แสดงผลลัพธ์จากการรันสคริปต์จากรูป ก-3

ภาคผนวก ข.

ไวยากรณ์ของภาษาไพธอน (Python Grammar)

ไวยากรณ์ของภาษาไพธอนทั้งหมด ซึ่งจะนำไปใช้อ้างอิง และช่วยในการออกแบบเร็กกูลาร์เอ็กเพรสชันและบีเอ็นเอฟสำหรับใช้กับอินเตอร์พรีเตอร์ไพธอนขนาดเล็กสำหรับบราวเซอร์ที่โปรแกรมได้บนโทรศัพท์มือถือ

identifier	::=	(letter "_") (letter digit "_")*
letter	::=	lowercase uppercase
lowercase	::=	"a"..."z"
uppercase	::=	"A"..."Z"
digit	::=	"0"..."9"
stringliteral	::=	[stringprefix](shortstring longstring)
stringprefix	::=	"r" "u" "ur" "R" "U" "UR" "Ur" "uR"
shortstring	::=	""" shortstringitem* """ ''' shortstringitem* '''
longstring	::=	""" longstringitem* """ ''' longstringitem* '''
shortstringitem	::=	shortstringchar escapeseq
longstringitem	::=	longstringchar escapeseq
shortstringchar	::=	<any ASCII character except "\" or newline or the quote>
longstringchar	::=	<any ASCII character except "\">
escapeseq	::=	"\" <any ASCII character>
longinteger	::=	integer ("I" "L")
integer	::=	decimalinteger octinteger hexinteger
decimalinteger	::=	nonzerodigit digit* "0"
octinteger	::=	"0" octdigit+
hexinteger	::=	"0" ("x" "X") hexdigit+
nonzerodigit	::=	"1"..."9"
octdigit	::=	"0"..."7"
hexdigit	::=	digit "a"..."f" "A"..."F"
floatnumber	::=	pointfloat exponentfloat
pointfloat	::=	[intpart] fraction intpart "."

exponentfloat	::=	(intpart pointfloat)
intpart	::=	digit+
fraction	::=	"." digit+
exponent	::=	("e" "E") ["+" "-"] digit+
imagnumber	::=	(floatnumber intpart) ("j" "J")
atom	::=	identifier literal enclosure
enclosure	::=	parenth_form list_display dict_display string_conversion
literal	::=	stringliteral integer longinteger floatnumber imagnumber
parenth_form	::=	"(" [expression_list] ")"
list_display	::=	"[" [listmaker] "]"
listmaker	::=	expression (list_for ("," expression)* [","])
list_iter	::=	list_for list_if
list_for	::=	"for" expression_list "in" testlist [list_iter]
list_if	::=	"if" test [list_iter]
dict_display	::=	"{" [key_datum_list] "}"
key_datum_list	::=	key_datum ("," key_datum)* [","]
key_datum	::=	expression ":" expression
string_conversion	::=	"'" expression_list "'"
primary	::=	atom attributeref subscription slicing call
attributeref	::=	primary "." identifier
subscription	::=	primary "[" expression_list "]"
slicing	::=	simple_slicing extended_slicing
simple_slicing	::=	primary "[" short_slice "]"
extended_slicing	::=	primary "[" slice_list "]"
slice_list	::=	slice_item ("," slice_item)* [","]
slice_item	::=	expression proper_slice ellipsis
proper_slice	::=	short_slice long_slice
short_slice	::=	[lower_bound] ":" [upper_bound]
long_slice	::=	short_slice ":" [stride]
lower_bound	::=	expression
upper_bound	::=	expression
stride	::=	expression
ellipsis	::=	"..."
call	::=	primary "(" [argument_list [","]] ")"

```

argument_list      ::=      positional_arguments ["," keyword_arguments]
                        ["," "*" expression] ["," "***" expression]
                        | keyword_arguments ["," "*" expression]
                        ["," "***" expression]
                        | "*" expression ["," "***" expression]
                        | "***" expression

positional_arguments ::=      expression ("," expression)*
keyword_arguments   ::=      keyword_item ("," keyword_item)*
keyword_item        ::=      identifier "=" expression
power               ::=      primary ["**" u_expr]
u_expr              ::=      power | "-" u_expr | "+" u_expr | "~" u_expr
m_expr              ::=      u_expr | m_expr "*" u_expr | m_expr "/" u_expr
                        | m_expr "/" u_expr | m_expr "%" u_expr
a_expr              ::=      m_expr | a_expr "+" m_expr | a_expr "-" m_expr
shift_expr          ::=      a_expr | shift_expr ("<<" | ">>") a_expr
and_expr            ::=      shift_expr | and_expr "&" shift_expr
xor_expr            ::=      and_expr | xor_expr "\textasciicircum" and_expr
or_expr             ::=      xor_expr | or_expr "|" xor_expr
comparison          ::=      or_expr (comp_operator or_expr)*
comp_operator        ::=      "<" | ">" | "==" | ">=" | "<=" | "<>" | "!=" | "is" ["not"]
                        | ["not"] "in"
expression          ::=      or_test | lambda_form
or_test             ::=      and_test | or_test "or" and_test
and_test            ::=      not_test | and_test "and" not_test
not_test            ::=      comparison | "not" not_test
lambda_form         ::=      "lambda" [parameter_list]: expression
expression_list      ::=      expression ("," expression)* [","]
simple_stmt          ::=      expression_stmt | assert_stmt | assignment_stmt |
                        augmented_assignment_stmt | pass_stmt | del_stmt |
                        print_stmt | return_stmt | yield_stmt | raise_stmt |
                        break_stmt | continue_stmt | import_stmt |
                        global_stmt | exec_stmt

```

```

expression_stmt ::= expression_list

assert_statement ::= "assert" expression ["," expression]

assignment_stmt ::= (target_list "=")+ expression_list

target_list ::= target ("," target)* [","]

target ::= identifier | "(" target_list ")" | "[" target_list "]" | attributeref |
          subscription | slicing

augmented_assignment_stmt ::= target augop expression_list

augop ::= "+" | "-" | "*" | "/" | "%" | "**=" | ">>=" | "<=<" | "&=" |
         "\textasciicircum=" | "|="

pass_stmt ::= "pass"

del_stmt ::= "del" target_list

print_stmt ::= "print" ( \optionalexpression ("," expression)* \optional"," |
                        ">\code>" expression \optional("," expression)+ \optional"," )

return_stmt ::= "return" [expression_list]

yield_stmt ::= "yield" expression_list

raise_stmt ::= "raise" [expression ["," expression ["," expression]]]

break_stmt ::= "break"

continue_stmt ::= "continue"

import_stmt ::= "import" module ["as" name]
              ( "," module ["as" name] )* |
              "from" module "import" identifier ["as" name]
              ( "," identifier ["as" name] )* |
              "from" module "import" "*"

module ::= (identifier ".")* identifier

global_stmt ::= "global" identifier ("," identifier)*

exec_stmt ::= "exec" expression ["in" expression ["," expression]]

compound_stmt ::= if_stmt | while_stmt | for_stmt | try_stmt | funcdef |
                  classdef

suite ::= stmt_list NEWLINE | NEWLINE INDENT statement+ DEDENT

statement ::= stmt_list NEWLINE | compound_stmt

stmt_list ::= simple_stmt ( ";" simple_stmt )* [ ";" ]

if_stmt ::= "if" expression ":" suite ( "elif" expression ":" suite )*
           ["else" ":" suite]

```

while_stmt ::=	"while" expression ":" suite ["else" ":" suite]
for_stmt ::=	"for" target_list "in" expression_list ":" suite ["else" ":" suite]
try_stmt ::=	try_exc_stmt try_fin_stmt
try_exc_stmt ::=	"try" ":" suite ("except" [expression ["," target]] ":" suite)+ ["else" ":" suite]
try_fin_stmt ::=	"try" ":" suite "finally" ":" suite
funcdef ::=	"def" funcname "(" [parameter_list] ")" ":" suite
parameter_list ::=	(defparameter ",")* ("*" identifier [, "*" identifier] "*" identifier defparameter [","])
defparameter ::=	parameter ["=" expression]
sublist ::=	parameter ("," parameter)* [","]
parameter ::=	identifier "(" sublist ")"
funcname ::=	identifier
classdef ::=	"class" classname [inheritance] ":" suite
inheritance ::=	"(" [expression_list] ")"
classname ::=	identifier
file_input ::=	(NEWLINE statement)*
interactive_input ::=	[stmt_list] NEWLINE compound_stmt NEWLINE
eval_input ::=	expression_list NEWLINE*
input_input ::=	expression_list NEWLINE

บรรณานุกรม

- [1] วีระศักดิ์ ชิงฉาว : “Fundamental of Java Programming Volume I”, SUM System Company Limited, 2543.
- [2] Mark Lutz and David Ascher : “Learning Python”, O’reilly & Associates Inc., CA, 1999.
- [3] H.M. Deitel, P.J. Deitel, J.P. Liperi and B.A. Wiedermann : “Python How To Program”, Prentice-Hall Inc., Upper saddle River, NJ 07458, 2002.
- [4] Vartan Piroumian : “Wireless J2ME Platform Programming”, Prentice Hall PTR, One Lak Street, Upper Saddle River, NJ 07458, 2002.
- [5] John W. Muchow : “Core J2ME Technology & MIDP”, Prentice Hall PTR, One Lak Street, Upper Saddle River, NJ 07458, 2002.
- [6] “Python Language Website”, <http://www.python.org>
- [7] Guido van Rossum and Fred L. Drake, Jr : “Python Reference Manual”, <http://www.python.org/doc/current/ref/>
- [8] “Bumble-Bee Software – Parser Generator – YACC and Lex for Windows”
<http://www.bumblebeesoftware.com>